Student ID: 24IT045

Chandubhai S Patel Institute of Technology

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement:

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.

Objective:

Run your full-stack Task Management application (React + Node.js/Express + MongoDB) using Docker containers with a single command.

Screenshots:

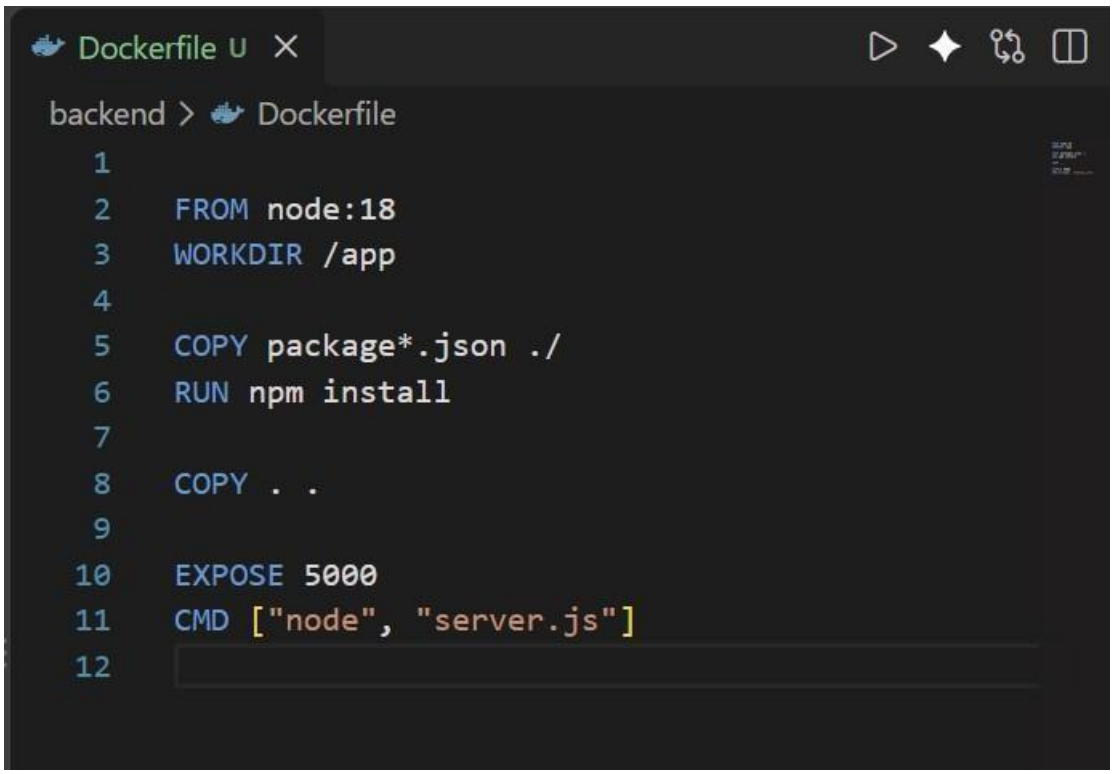
Step 1: Verify Docker is installed

```
C:\Users\24IT022>docker --version
Docker version 29.8.2, build 7fc2dff

C:\Users\24IT022>docker compose version
Docker Compose version v5.5.1

C:\Users\24IT022>docker info
Client:
 Version:    29.8.2
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.144.0
    Path:     C:\Users\24IT022\.docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.31.0
    Path:     C:\Users\24IT022\.docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.37.2
    Path:     C:\Users\24IT022\.docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.5.1
    Path:     C:\Users\24IT022\.docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Users\24IT022\.docker\cli-plugins\docker-debug.exe
```

Step 2: Create the backend Dockerfile



Step 3: Create the frontend Dockerfile

Dockerfile U

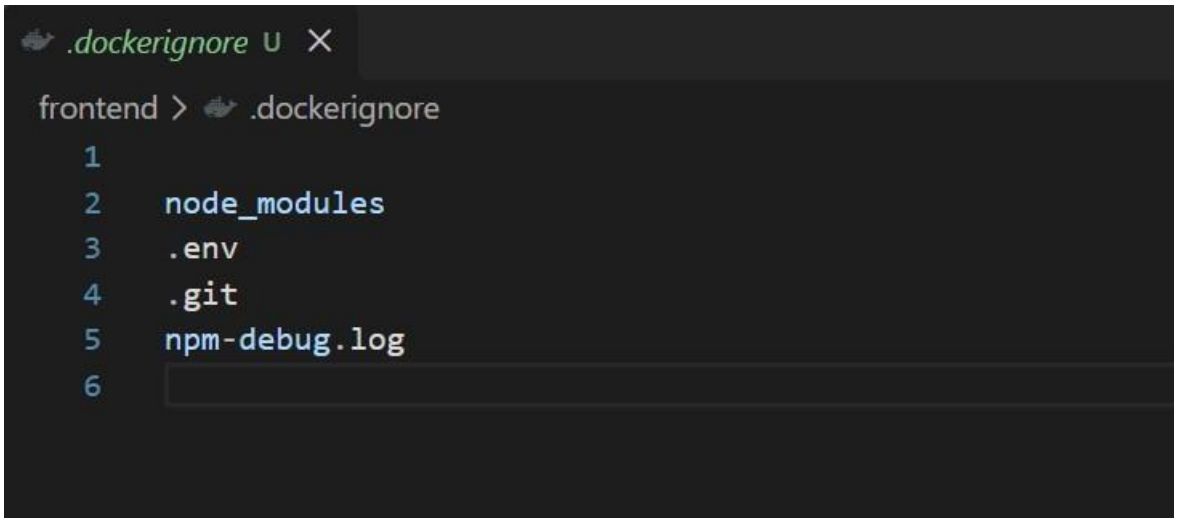
frontend > Dockerfile

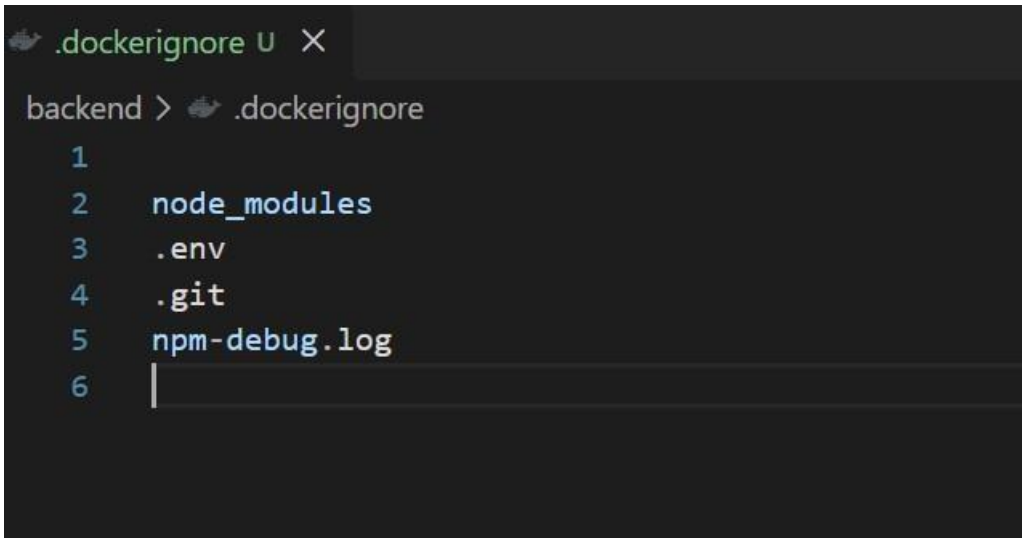
```
1
2   FROM node:18 AS build
3   WORKDIR /app
4
5   COPY package*.json ./
6   RUN npm install
7
8   COPY . .
9   RUN npm run build
10
11  FROM node:18 AS production
12  WORKDIR /app
13
14  COPY --from=build /app/package*.json ./
15  COPY --from=build /app/node_modules ./node_modules
16  COPY --from=build /app/dist ./dist
17
18  EXPOSE 5173
19  CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0", "--port", "5173"]
```

Step 4: Configure the MongoDB connection

```
29  // 4. Connect to MongoDB Database
30
31  mongoose.connect(
32    process.env.MONGO_URI || "mongodb://127.0.0.1:27017/management_system_db"
33  )
34  .then(() => console.log("MongoDB connected"))
35  .catch((err) => console.error("MongoDB connection failed:", err));
36
```

Step 5: Create .dockerignore files





Step 6: Create docker-compose.yml

Student ID: 24IT045

docker-compose.yml U

docker-compose.yml

```yaml

services:
  frontend:
    build:
      context: ./frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build:
      context: ./backend
    ports:
      - "5000:5000"
    environment:
      MONGO_URI: mongodb://127.0.0.1:27017/management_system_db
    depends_on:
      mongodb:
        condition: service_healthy
    networks:
      - app-network
```

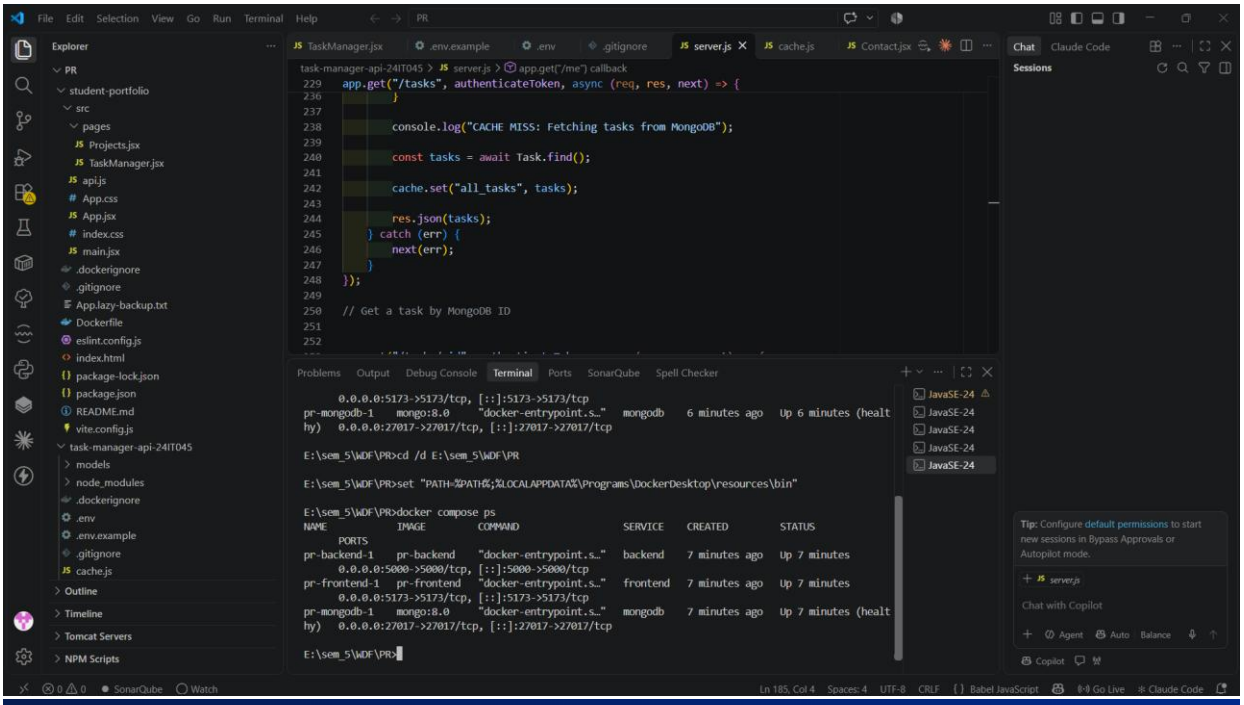
Student ID: 24IT045

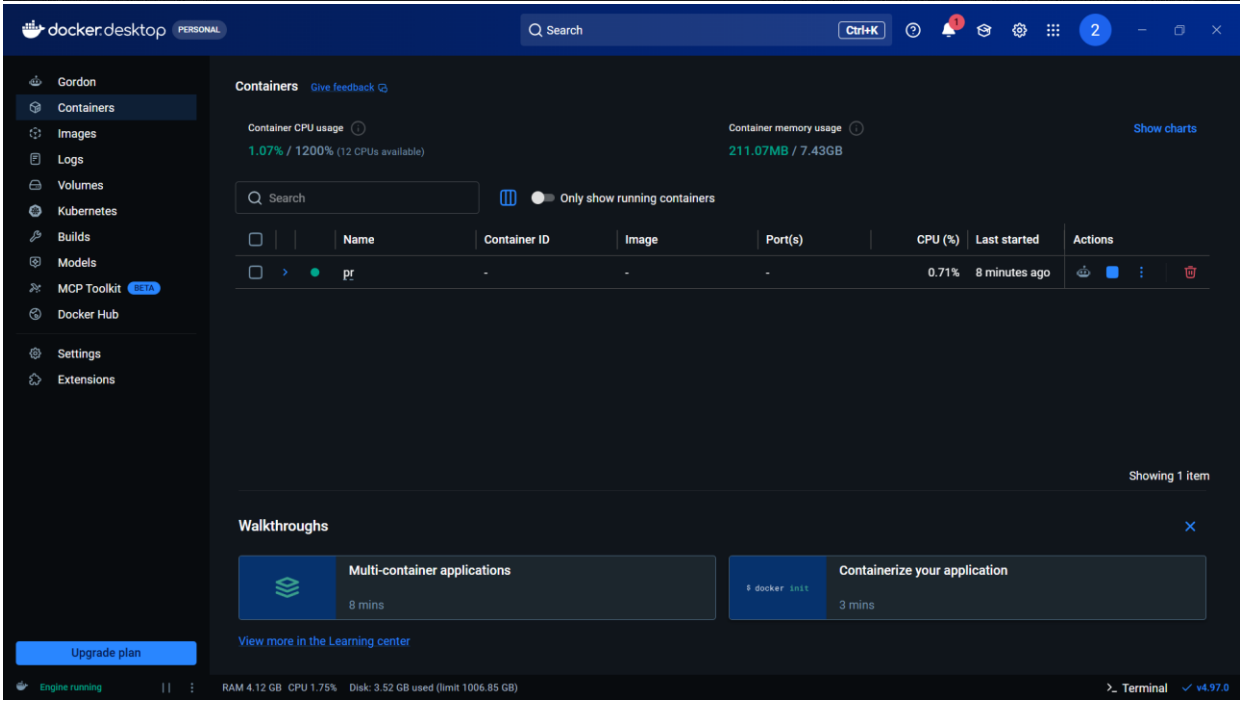
```yaml
  mongodb:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test:
        - CMD
        - mongosh
        - --quiet
        - --eval
        - "db.adminCommand('ping').ok"
      interval: 5s
      timeout: 3s
      retries: 10
      start_period: 10s
    networks:
      - app-network
networks:
  app-network:
    driver: bridge
volumes:
  mongo-data:

```

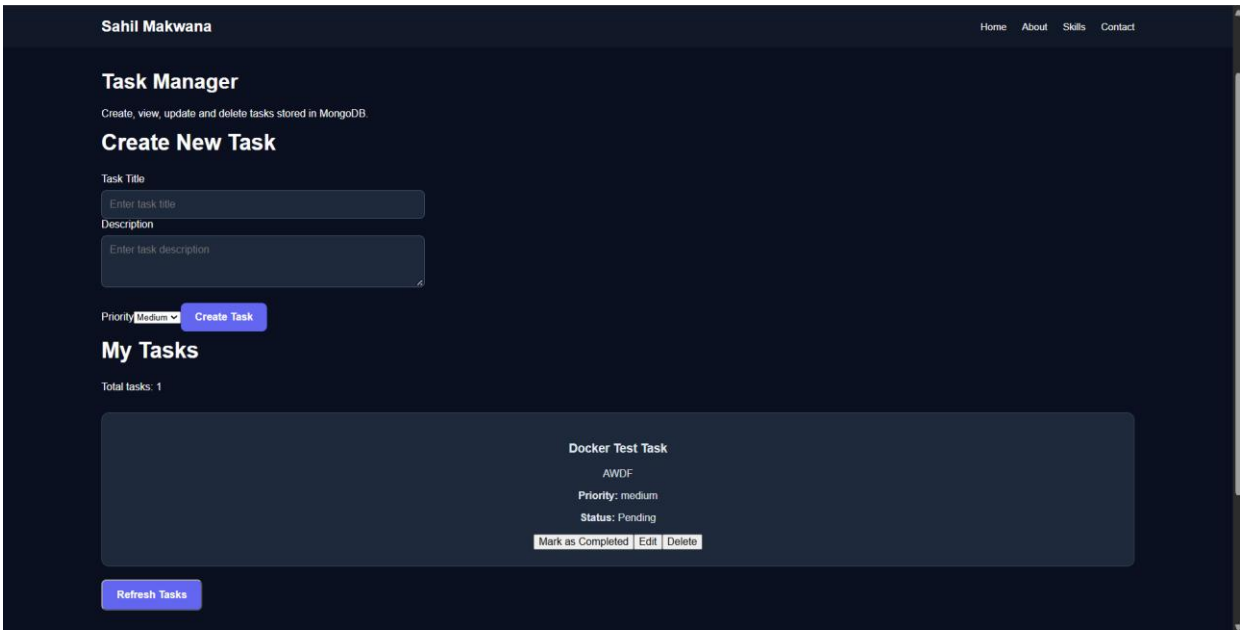
Student ID: 24IT045

Step 7: Verify all three containers





Student ID: 24IT045



Github Repo Link: https://github.com/sahilmak1602/Task-Manager-Portfolio-24IT045.git